

Unit 4 Open Source Software (OSS) & Patents

Open Source Ecosystem – Overview

- Open Source Software (OSS) is now the backbone of modern software industry
- 96–99 % of all commercial software contains open-source components
- All cloud infrastructure (AWS, Azure, GCP), Android, Linux kernel, Kubernetes, TensorFlow, VS Code, Chrome, Firefox, PostgreSQL, Apache, Nginx, Docker, etc. are open source or heavily depend on OSS
- Market share: Linux > 90 % of cloud servers, > 70 % of smartphones (Android), > 80 % of embedded devices
- Fortune 500 companies: All of them use OSS internally and many contribute back

Background: Laws and Open Source



Copyright protects creative, intellectual and artistic works – including software

- The moment you write code, it is automatically copyrighted in 179+ countries (Berne Convention).
- Copyright gives the author 6 exclusive rights: reproduction, distribution, derivative works, public performance, display, and public digital performance.
- Without a license, nobody else can legally copy, modify, or distribute your code.

Alternative: public domain (nobody may claim exclusive property rights)

- Author voluntarily gives up all rights (e.g., CC0, Unlicense) or copyright expires (life + 70 years in most countries).
- Example: SQLite is public domain — anyone can embed it even in fully closed-source products.

Trademark protects the name and logo of a product

- You can use Mozilla Firefox code (MPL license), but you cannot call your modified browser “Firefox” or use the fox logo without permission.
- Famous case: Iceweasel (Debian) vs Firefox naming dispute.

OSS is generally copyrighted, with copyright retained by contributors or assigned to entity that maintains it

- Linux kernel copyrights are held by thousands of individuals and companies, but all agree to the GPL v2 license.

Copyright holder can grant a license for use, placing restrictions on how it can be used (perhaps for a fee)

- Proprietary → restrictive license + fee (Microsoft Windows)
- Open source → liberal license + usually free of cost (Linux, Apache)

What is Open-Source Software (OSS)?

Open-source software is software with publicly available code. This means that **anyone can not only inspect the source code but also modify it and enhance it.**



1. Source code availability
 - “The program must include source code, and must allow distribution in source code as well as compiled form.”
 - Example: You can download the exact Linux kernel source that runs on Google, AWS, and 96% of top supercomputers.
2. Right to modify and create derivative works
 - You can fix bugs, add features, or even completely rewrite parts.
 - Real example: Red Hat, Ubuntu, SUSE, Alibaba Cloud all modify the same Linux kernel for their needs.
3. (Often) Right to redistribute derivative works
 - Some licenses (MIT, Apache, BSD) allow you to sell your modified version.
 - Others (GPL) allow redistribution but force you to give recipients the same rights (“share-alike”).

Analogy:

Open source is like getting a Lego set with the full instruction manual and permission to rebuild it however you want and even sell your new designs. Proprietary is like getting a finished Lego model glued together with a “Do not open” sticker.

Contrast with proprietary software: a black box

- **Open-Source:**

Open-source software comes with licenses that let users freely use, modify, and share the software. However, there are different types of open-source licenses with varying rules. For instance, licenses like the GPL require that any modified versions also remain open-source, while others like MIT or Apache don't have such requirements. It's important for businesses to understand these licenses to avoid any potential legal issues.

COMPARISON OF OPEN-SOURCE AND PROPRIETARY SOFTWARE		
	proprietary software	open-source software
licence free		
maintenance support	commercial support	community or commercial support, if available
source code access	no access	access, modification possible

- **Proprietary:**

Proprietary software is licensed under strict terms, often limiting how the software can be used, shared, or modified. Users must comply with the terms and conditions set by the vendor, and breaking them can result in penalties. The legal considerations are more straightforward, but the

restrictive nature of proprietary licenses can limit flexibility for users.

- Intention is “use only” — not to study, improve, or fix.
- You receive only the compiled binary (machine code) — impossible to read or modify.
- EULA (End User License Agreement) examples:
 - “You shall not reverse engineer, decompile, or disassemble the software” (almost every EULA).
 - Oracle Java, Adobe Photoshop, and even some games forbid benchmarking without permission!
- Real-world consequence: 2017 Equifax breach — they couldn't patch Apache Struts fast enough partly because they were using closed-source tools and couldn't audit internals easily.

Analogy:

Proprietary software is like buying a car with the hood welded shut. You can drive it, but you can't fix it yourself or see how the engine works.

Early open source: UNIX to BSD

1. Hardware was not standardized

- 1970s: Every computer vendor (IBM, DEC, HP, etc.) had completely different CPUs and architecture.
- Software written for one machine rarely worked on another.
- Companies made money from hardware → software was just a way to sell the machine.

2. Unix born at AT&T Bell Labs

- 1969–1974: Ken Thompson and Dennis Ritchie created Unix.
- Written in a new portable language called C → same source code could run on different machines.
- Very clean, small, and powerful → universities loved it.

3. AT&T couldn't sell software

- Due to 1956 U.S. government rule, AT&T was a regulated phone monopoly.
- They were legally not allowed to be in the computer business.
- So they gave Unix licenses very cheaply (sometimes almost free) to universities and research labs → always with full source code.

4. 1978: UC Berkeley enters the story

- Bill Joy and team started adding major features: TCP/IP (Internet!), virtual memory, better file system, vi editor, etc.
- They bundled everything and released it as BSD (Berkeley Software Distribution).
- Anyone with a Unix license could get BSD tapes with full source → modify and use freely.

5. 1982–83: Everything changes

- U.S. government breaks up AT&T monopoly → AT&T is now allowed to sell software.
- AT&T immediately stops giving source code → starts charging huge fees for Unix.
- Berkeley says “fine” and keeps improving BSD independently.

The BSD License is Permissive

Explain the original 4-clause BSD license (1990):

Only two practical requirements:

1. Keep the copyright notice and credit University of California.
2. No liability — “use at your own risk”.

Later simplified to 3-clause and 2-clause (removed advertising clause).

Today's BSD license is only ~150 words — one of the shortest and most business-friendly.

Real impact:

- Apple macOS and iOS are based on BSD (Darwin).
- PlayStation 4/5, Nintendo Switch firmware — all contain large amounts of BSD code.

UNIX to GNU's Not Unix

1. **1978** UC Berkeley begins distributing their own heavily improved version of Unix → called BSD → Full source code, freely modifiable → very popular in universities and startups.
2. **1983 – The Big Shock** AT&T is finally freed from the old monopoly rules → they can now sell software for profit. → Overnight, AT&T changes Unix licensing:
 - No more source code in cheap licenses
 - Very high prices (thousands to lakhs of rupees) → Universities and researchers suddenly lose access to new Unix versions.
3. **Immediate Effect** All big computer companies start selling their own commercial Unix:
 - AT&T → System V
 - HP → HP-UX
 - Sun → SunOS/Solaris
 - IBM → AIX
 - SGI → IRIX → Each version slightly different → “Unix wars” and fragmentation”.
4. **Also 1983 – Richard Stallman's Reaction** Richard Stallman (brilliant MIT hacker) gets furious: “Proprietary software is immoral. Users should have freedom.” → September 1983: He announces the GNU Project on the net.unix-wizards mailing list: “Starting this Thanksgiving I am going to write a complete Unix-compatible software system called GNU (GNU's Not Unix), and give it away free to everyone who can use it.”

Free software as a Philosophy

Richard Stallman's Four Freedoms (memorize this — it comes in exams):

- Freedom 0: Run the program as you wish, for any purpose.
- Freedom 1: Study and change the source code (requires source availability).
- Freedom 2: Redistribute exact copies to help your neighbor.
- Freedom 3: Distribute your modified versions so the community benefits.

“Free as in speech, not as in beer” (libre, not gratis)

GPL = Copyleft (viral/share-alike)

- If you modify and distribute GPL code, your entire product must also be GPL.
- Prevents companies from taking open source and making it proprietary again.

Alternative: Permissive (BSD/MIT/Apache)

- “Do whatever you want, just keep the copyright notice and don’t sue me.”

Copyleft vs. Permissive

Question	Permissive (MIT, BSD, Apache)	Copyleft (GPL, AGPL)
Can I use it in my closed-source product?	Yes	No (effectively)
Can I sell my modified version?	Yes	Yes, but must give source
Does it force others to share code?	No	Yes (“viral”)
Preferred by corporations	Yes (React, .NET, Android)	Less (Linux kernel is exception)
Protects the commons	Weakly	Strongly

1. GNU/Linux (1991–Today)

- 1983: Stallman started GNU to make a fully free Unix-like OS.
- He and volunteers wrote almost everything: compiler (GCC), editor (Emacs), shell, libraries, etc.
- Only one big piece was missing in 1991 → the kernel (heart of OS).
- 1991: Linus Torvalds released Linux kernel (free + open source).
- People combined GNU tools + Linux kernel → GNU/Linux (we just say “Linux”).
- 1990s–2000s: Internet boom → everyone wanted cheap/free OS for servers → Linux won.
- Companies (Red Hat, SUSE, Canonical) give free Linux + paid support.
- Red Hat alone makes > \$4 billion/year. IBM bought Red Hat for \$34 billion.
Today: 96% of top supercomputers, almost all cloud servers, Android phones → all run Linux.

2. Netscape’s open-source gambit

- Early-mid 1990s: Netscape Navigator was the #1 browser (90%+ market).
- 1998: Microsoft gave Internet Explorer free with Windows → Netscape started losing fast.
- Jan 1998: Netscape did something crazy → released the full browser source code openly.
- This was the first big company to open-source its main product.
- The released code became Mozilla Firefox → still alive today.

3. Netscape creates a new license and model

- Before 1998: Most open-source projects used GPL (Stallman style).
- Netscape didn't want GPL → they made their own license (MPL).
- 1998: Eric Raymond wrote "The Cathedral and the Bazaar" → showed open source can be better and faster.
- Same year: "Open Source" term was born (instead of "Free Software").
- 1998: Netscape move + Eric's paper → big companies started trusting open source.
- Result: Apache, Linux, Firefox, Android, Chrome, VS Code, etc. all exploded after 1998.

The Bazaar won

- Cathedral
 - Developed centrally by a core group of members
 - Available for all once complete (or at releases)
 - Examples: GNU Emacs, GCC (back in the 1990s)
 - "Sort-of" examples today: Chrome, IntelliJ
- Bazaar
 - Developed openly and organically
 - Wide participation (in theory, anyone can contribute)
 - Examples: Linux

OSS has many stakeholders / contributors

- **Core members**
 - Small, highly trusted group who have direct write access to the main repository.
 - Often (but not always) includes the original creator(s) of the project.
 - They can merge or reject code themselves.
 - Roles are sometimes split further into administrators (who manage permissions) and lead developers.
- **External contributors**
 - People outside the core team.
 - They open bug reports, suggest features, and discuss issues.
 - They submit actual code or documentation through pull requests / merge requests.
- **Other supporters**
 - Beta testers and everyday users who report real-world bugs.
 - Companies or individuals who give money or provide servers, CI/CD, websites, etc.
 - Steering committees or public commenters who write standards and RFCs (Request for Comments).
- **Spin-offs**
 - Separate teams that take the original code and maintain their own version (a fork) – e.g., MariaDB forked from MySQL, Jenkins X from Jenkins.

Contributing processes

- Mature open-source projects almost always have strict contribution guidelines (look for a file called CONTRIBUTING.md or similar in the repository).

Common requirements that projects enforce:

- **Coding style** – exact rules for indentation, braces, naming; enforced by linters and pre-commit hooks.
- Inclusion of test cases with every new feature or bug fix – tests must pass 100%.
- Minimum number of code reviews/approvals from core developers before merging.
- Standards for documentation – every public function or API must be documented.
- Contributor licensing agreements (CLA or DCO) – legal paper saying you allow your code to be used under the project's license (explained in detail later).

Governance

- Some important OSS projects are fully managed and controlled by for-profit companies Examples given on slide:
 - Chromium → Google
 - Moby (Docker engine) → Docker Inc.
 - Ubuntu → Canonical
 - TensorFlow → Google
 - PyTorch → Meta
 - Java → Oracle
- In these projects, contributors are a mixture of paid company employees + outside volunteers. The company usually pays for websites, build servers, testing farms, marketing, etc.
- Many other major OSS projects are governed by non-profit foundations or loose community structures Examples: Apache projects (Hadoop, Spark, Kafka, Tomcat etc.) → Apache Software Foundation (ASF) Firefox → Mozilla Foundation Python → Python Software Foundation (PSF) NumPy, SciPy → community governed
- Foundations pay for servers, legal help, and conferences through corporate sponsorships and donations.
- Long-term roadmap is decided either collaboratively (Apache style) or by a “benevolent dictator for life” (Linux → Linus Torvalds, Python → Guido van Rossum earlier).
- Even community-owned projects today receive huge money from Big Tech (e.g., Google is the biggest single donor to Mozilla).

Corporate outlook towards open-source has evolved over the years

- 2001 – Microsoft internal Halloween documents + CEO Steve Ballmer publicly called Linux and GPL a “cancer” and said open source was un-American and destroys intellectual property.

- 2014 – New CEO Satya Nadella openly declared “Microsoft love Linux” on stage.
- Today – Microsoft is the largest corporate contributor to the Linux kernel, owns GitHub (world’s biggest open-source platform), runs >60% of Azure virtual machines on Linux, and makes billions from open-source-based services.

Use of open-source software within companies

Key questions every company must ask before using OSS:

- **Is the license compatible with our intended use?**
 - Can we use it internally? In a product we sell? In a SaaS service?
 - Permissive (MIT, Apache, BSD) → almost always yes
 - Strong copyleft (GPL, AGPL) → may force the whole product to become open source
- **How will we handle versioning and updates?**
 - Do we pin (freeze) to one exact version forever, or keep upgrading?
- **Does every internal project declare its own versioned dependency or do we all agree on using one fixed (e.g., latest) version?**
 - Some teams lock to “OSS version X” forever
 - Others always take the newest → creates “dependency hell” when two teams need different versions of the same library
- **How to resolve subtle or testing updates and breaking changes?**
 - A third-party dependency who are responsible for testing updates and bearing allowable versions.
 - Internal forks are useful but hard to keep in sync with upstream changes.
- ****One option: Assign an internal owner who keeps an internal fork up-to-date with upstream.**
 - Another option: **Contribute all customizations and bug fixes back** to the upstream project → keeps the fork minimal and clean.
- **Security risks? Supply chain attacks on the rise.**
 - 2021 Log4Shell, 2024 xz-utils backdoor, 2023 PyPI/ npm malware → companies now scan every dependency (SBOM – Software Bill of Materials) and watch for vulnerabilities daily.

Modern software is extremely interdependent. One angry developer, one backdoor, or one unmaintained library can affect billions of users and billions of dollars.

Software Licenses

1. Which license to choose?

- Official choosealicense.com decision tree (by GitHub) → Asks simple questions to recommend the right license:
 - Do you want to allow commercial use?
 - Do you want to allow modifications?
 - Do you want to force others to share their changes?
 - Do you want to allow use without sharing source? → Depending on answers, it suggests MIT, Apache 2.0, GPL, etc.

2. GNU General Public License: The Copyleft License

	 Copyright	 Copyleft	 Permissive	 Creative Commons
What is a user allowed to do with the code?	What creator dictates	What user wants under certain rules	What user wants with a few restrictions	What user wants without restrictions
Clause of the use	As creator dictates	Derivative work must be attributed to creator, open-source and copyleft	Derivative work must be attributed to a creator	Derivative work must be attributed to a creator
Source code	As creator dictates	Must be open	Don't have to be open	No specific terms about the distribution of source code
Is creator liable for bugs?	✓ YES	✓ YES	✗ NO	✗ NO
Re-licensing	As creator dictates	Derivative work cannot be released as proprietary software	Derivative work can be released under another license or as proprietary software	Derivative work can be released under another license or as proprietary software
Commercial restrictions	As creator dictates	Permitted	Permitted	Permitted

Compare copyright, copyleft, permissive, creative commons with examples

- Core philosophy: “Nobody should be restricted by the software they use.”
- Four freedoms every user must have:
 1. Freedom to use the software for any purpose
 2. Freedom to study and change the source code
 3. Freedom to share the original software
 4. Freedom to share your modified versions
- Strong rule: Any modifications or derivative works must be released under the exact same GPL license (this is called copyleft or “viral” effect).

3. Risks of “copyleft” licenses (example: GNU GPL)

- Requires derivative works also under GPL
- This is intentional!

- Result:
 - If you link or include GPL code inside your proprietary or differently-licensed program → the whole combined program legally becomes GPL
- Viral effect of polluting everything else with GPL requirement
- Most companies will avoid GPL code in proprietary products because they don't want to be forced to open-source their own code
- Many engineers are not allowed to even copy-paste GPL snippets from GitHub

4. Lesser GNU Public License (LGPL)

- Designed specifically so software can be used as a library
- Almost same as GPL but does not treat dynamic linking as “derivative work”
- Therefore proprietary/commercial programs can safely use LGPL libraries licensed under LGPL without becoming GPL themselves (as long as they don't modify the LGPL library itself)
- Famous example: Oracle JDK uses LGPL for some parts with the classpath exception

5. MIT License

- Extremely simple and very commercial-friendly
- Only four short requirements:
 - Must keep the original copyright notice
 - Must keep the license text
 - Software is provided as-is
 - Authors are not liable for anything
- No other restrictions → you can do anything (sell it, close the source, modify, sublicense)

6. Apache License 2.0

- Very similar to MIT in permissiveness
- Not copyleft
- Extra features compared to MIT:
 - Explicitly grants patent rights (protects users from patent lawsuits)
 - Does not allow you to use the original project's trademarks
 - Does not force modifications to keep the same license

7. BSD License (2-clause or 3-clause)

- No liability and provided as-is
- Only requirement: keep the copyright statement in both source code and binaries
- The copyright holder does not endorse any modified versions unless explicitly agreed in writing
- Extremely permissive – widely used in operating systems (macOS, PlayStation, Nintendo Switch all contain BSD code)

8. Creative Commons (CC)

- Mainly designed for creative content (images, articles, videos, datasets, slides, music), not for software
- Common variants:
 - CC-BY → attribution only
 - CC-BY-SA → attribution + share-alike (like GPL for non-code)
 - CC-BY-ND → attribution + no derivatives allowed
- Never use CC licenses for source code (OSI and FSF both say it's wrong for software)
-

Dual License Business Model

- Company releases the **exact same software under two licenses at the same time**:
 - GPL (or AGPL) → completely free, but forces anyone who uses it in a product to open-source their whole application
 - Separate paid commercial license → business-friendly (no copyleft)
- Classic real example on the slide: **MySQL**
 - Free community version = GPL
 - If a company embeds MySQL in their closed-source product → they must buy a commercial license from MySQL AB (now Oracle)
 - Price example: companies paid \$2,000 to \$10,000+ annually per server just to avoid GPL obligations
- Other famous dual-licensed projects: Qt, MongoDB (SSPL + commercial), Redis (RSAL + commercial after 2024)

Risk: Incompatible Licenses

- Example on the slide: **OpenOffice** → **LibreOffice**
 - Sun Microsystems open-sourced StarOffice as OpenOffice.org under LGPL
 - 2010: Oracle bought Sun → many contributors feared Oracle would kill or close the project
 - Oracle temporarily stopped active development
 - Community forked the code and created **LibreOffice**
 - Later Oracle donated the OpenOffice trademark and remaining code to Apache Foundation
 - Result today:
 - Apache OpenOffice (Apache License 2.0)
 - LibreOffice (MPL 2.0 + LGPL) → The two projects **cannot merge code freely** anymore because Apache 2.0 and MPL/LGPL are incompatible in some directions → permanent split caused purely by license conflict

Copyright vs. Intellectual Property (IP)

- **Copyright**
 - Automatically protects books, music, art, source code the moment it is created
 - Covers the exact expression, not the idea
 - No registration needed, lasts life + 70 years in most countries
- **IP and Patents**
 - Protect ideas or ways of solving problems, not just the expression
 - Examples: machine designs, drug formulas, UI gestures, one-click buying
 - Expensive and slow to get (20-year monopoly)
 - Can be licensed or sold for money
- **Copyright covers particular expressions of some work**
 - Books, songs, paintings, source code → automatic protection
- Exceptions: trivial works and pure ideas are not protected by copyright

Contributor Licensing Agreements (CLA)

- Many big projects (Google, Meta, Apache, Linux Foundation, etc.) force every contributor to sign a CLA before accepting code
- Two main types:
 - Contributor assigns copyright of their patch to the project/foundation
 - Or contributor grants very wide license (including future changes)
- Reason written on the slide: “Without this, you own the copyright and IP for even small bug fixes and that can cause them legal headaches in the future”
→ Example: if a contributor later sues or changes mind, the project can be in danger
-

The 4 Main Types of Intellectual Property (summary table)

1. Copyrights

Copyrights protect original works of authorship, such as literature, music, and art. They do not apply to inventions or functional objects like an umbrella. Therefore, copyrights would not be appropriate for protecting Daisy's novel invention.

2. Patents

Patents are the most appropriate IP right for protecting inventions, particularly those that are novel, non-obvious, and useful. Daisy's umbrella, which can be folded six times and easily carried, qualifies as an invention. By obtaining a patent, Daisy would secure exclusive rights to make, use, and sell the umbrella for a limited period, typically 20 years from the filing date.

3. Trade Marks

Trademarks protect brand names, logos, slogans, and other identifiers that distinguish the goods or services of one entity from those of others. They do not

protect the functional aspects or design of an invention like Daisy's umbrella. Thus, trademarks are not suitable for protecting this type of innovation.

4. Trade Secrets

Trade secrets protect confidential business information, such as formulas, processes, or strategies, that gives a competitive advantage. While trade secrets could theoretically protect the method of making the umbrella if kept secret, they are not the best fit for protecting the invention itself. A patent would be more appropriate for this purpose since it would offer broader protection and allow Daisy to enforce her rights legally.

IP Type	Validity	Definition	Example	Rights Granted
Patents	20 years from filing date	Exclusive right granted for an invention that is new, non-obvious, and useful.	A new pharmaceutical drug, a novel manufacturing process, or a unique mechanical device.	The right to exclude others from making, using, selling, offering for sale, or importing the patented invention.
Copyrights	Life of the author + 60 years	Exclusive rights granted to the creators of original works of authorship such as books, songs, movies, and computer programs.	A novel, a musical composition, a feature film, or the source code of a software application.	The right to reproduce, distribute, display, perform, and make derivative works of the copyrighted material.
Trademarks	Renewable 10-year terms	Distinctive signs that identify the source of goods or services.	The Nike "Swoosh" logo, the Coca-Cola brand name, or the McDonald's "Golden Arches".	The right to use the mark to identify and distinguish the goods or services, and to prevent others from using similar marks that would likely cause consumer confusion.
Trade Secrets	Indefinite, as long as the information remains secret	Confidential business information that provides a competitive advantage.	The formula for Coca-Cola, the recipe for KFC's fried chicken, or the algorithm used by a search engine.	The right to maintain the secrecy of the information and to take legal action against those who misappropriate the trade secret.
Industrial Designs	10 years, renewable for	The ornamental or aesthetic aspects of a product.	The distinctive shape of a mobile phone, the unique design of a car's headlights, or the	The right to prevent others from making, selling, or importing products that copy the registered design.

	another 5 years		decorative pattern on a piece of furniture.	
--	--------------------	--	--	--

Software Patents



What is a Software Patent?

A government-granted 20-year monopoly on a specific **idea or method** performed by software

Not the code itself (that's protected by copyright), but the **invention/algorithm/process**

Example:

- Amazon's "1-Click Buying" patent (US 5,960,411)
- GIF image format's LZW compression patent
- MP3 audio encoding patents
- Slide-to-unlock, pull-to-refresh, rounded corners (Apple vs Samsung cases)

What is a patent? New. Useful. Non-obvious.

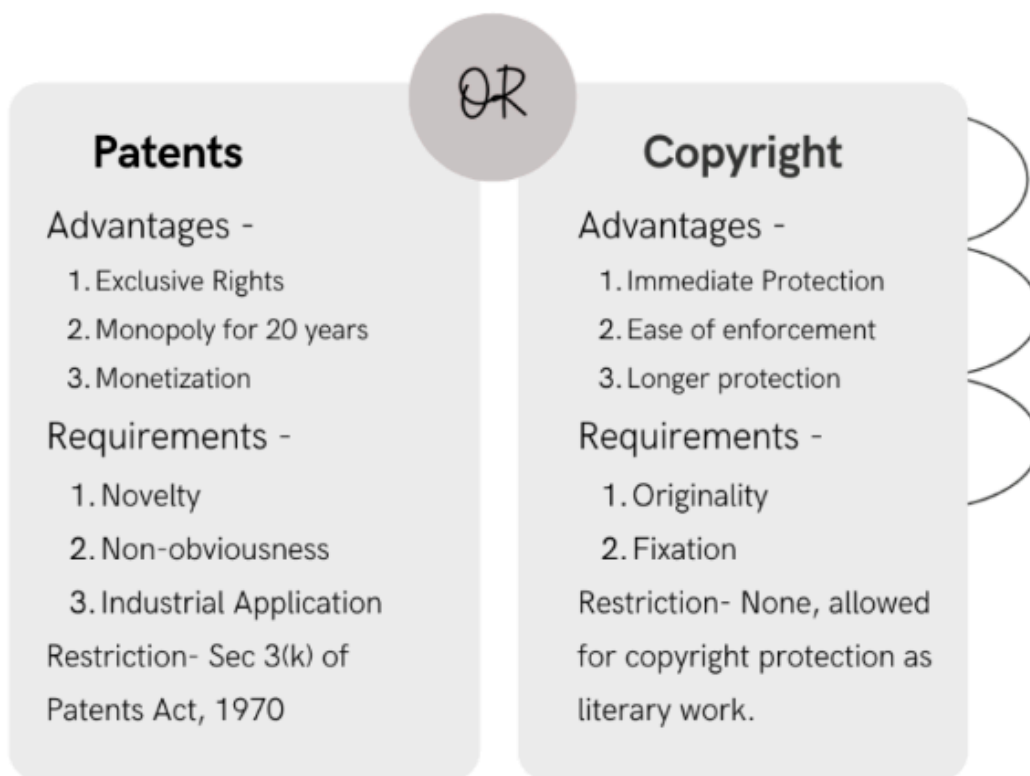
- Official definition (WIPO + almost all countries): "A patent is an exclusive right granted for an invention, which is a product or a process that provides, in general, provides a new way of doing something, or offers a new technical solution to a problem."
- To get a patent, the invention must be:
 - **New** (never published anywhere before)
 - **Useful** (actually works, has industrial application)
 - **Non-obvious** (not trivial for an expert in the field)
- Full technical information about the invention must be publicly disclosed in the patent document.

What rights do patents grant?

- Patents do **not** give you the positive right to make, use, or sell your own invention (you might still be blocked by someone else's earlier patent).
- Patents give you the **negative right** to exclude others from:
 - making
 - using
 - selling
 - importing the patented invention for **20 years** from filing date.
- You can earn money through:
 - stop or sue others
 - licensing and royalties (e.g., Qualcomm makes billions only from patent licensing)

What's the difference? Patents vs. Copyright

What is preferable for software?



- Copyrights cover the **details of expression** of a work → the exact source code, book text, song melody
- Copyrights do **not** cover any ideas
- Patents only cover **ideas and the use of ideas** → one patent can block every possible implementation of the same algorithm
- Copyrights happen **automatically**

- Patents are issued by a patent office only after long examination of a formal application (1–5 years, expensive)

Why do patents exist?

- Encourage inventors to **publicly disclose** inventions instead of keeping them secret
- Reward invention and creativity with temporary monopoly
- Protect investment of huge R&D money
- Encourage the market to “design around” the patent → creates more innovation
- Protect small companies/start-ups from being instantly copied by giants

Patent or not? (Real examples from slide – all of these have actually been filed or granted somewhere)

- Running bingo on a computer → **patented**
- Using a computer to help users plan meals while achieving diet goals → **patented**
- Using a computer to order a pizza with customized toppings → **patented**
- Prompting a user before establishing a new network connection → **patented**
- Automatically notifying users when an item is picked up or delivered → **patented**
- Using computer network to ask people to complete tasks and then wait for them to do them → **patented**
- Using SMS to perform tasks (e.g., checking bank balance) → **patented**
- Selecting ALL images in a CAPTCHA that match a given text → **patented** (Google reCAPTCHA patent)

Problem: Incompatibility

- GIF (1987) used LZW compression → Unisys suddenly enforced its patent in 1994–1995 → demanded royalties from every website and software that created GIFs → Entire internet panicked → PNG (Portable Network Graphics) was invented in 1995 purely as a patent-free replacement
- MP3 format was covered by dozens of patents owned by Fraunhofer, Thomson, Alcatel-Lucent, etc. → Royalties had to be paid for every encoder/decoder until patents expired around 2017 → Opus (2012) was deliberately engineered from scratch with only unpatented or royalty-free techniques so no one ever has to pay again
- H.264 & H.265 (HEVC) are controlled by patent pools (MPEG-LA, HEVC Advance, Velos) → licensing fees can be thousands to tens of thousands of dollars per year per company → Google, Amazon, Netflix, Mozilla, Intel,

Microsoft, Apple, etc. formed the Alliance for Open Media and spent billions of dollars and 8+ years to create AV1 (2018) – a completely royalty-free modern video codec → Moral: patents force the entire industry to reinvent the wheel again and again instead of improving the best existing solution.

Problem: Independent discovery doesn't matter!

- Full John Carmack quote (2012): “The idea that I can be presented with a problem, set out to logically solve it with the tools at hand, and wind up with a program that could not be legally used because someone else followed the same logical steps some years ago and filed for a patent on it is horrifying.”
- Real example: – British Telecom claimed in 2000 that it owned the patent on hyperlinks themselves (US patent 4,873,662 from 1989) → sued ISP Prodigy for millions (case eventually lost, but cost millions to defend) – If you reinvent a patented algorithm in your garage without ever seeing the patent → you are still infringing and can be sued for damages + forced to stop.

Problem: Only large organizations benefit

- Challenging a single U.S. software patent in court costs \$1.5 million to \$5 million on average (American Intellectual Property Law Association 2023 report)
- Taking a patent fight international (Europe, China, Japan) easily reaches \$15–45 million
- Patent trolls (non-practising entities) file 60–70 % of all patent lawsuits in the U.S.
- Small startups and open-source projects almost never have this money → they either:
 - pay the demanded license fee,
 - remove the feature,
 - or shut down completely → The patent system that was supposed to “protect the little guy” actually protects only companies big enough to have a huge patent war-chest.

Problem: Innovation is stifled

- Startup developer quote (from EFF “Defend Innovation” paper): “As a developer for a small startup, absurd software patents are a constant worry. Stories abound of people like us getting pressured out of existence over the use of incredibly vague, basic interface elements and system components.”
- Patents are deliberately written in extremely broad language: – “A method of conducting electronic commerce over a network” – “Displaying information in a scrollable window” – “Progress bar indicating download status” → These cover almost every modern app
- Result → developers practise “defensive programming”: – avoid shopping carts, progress bars, rounded corners, swipe gestures, in-app purchases, etc. – or they copy Apple/Google designs (because big companies cross-license each other) → Real innovation is pushed underground or never happens.

This American Life: When Patents Attack! (2011 NPR episode)

- Patent troll “Project Paperless” claimed to own “scanning a document and emailing it” → sent demand letters to thousands of small businesses asking for \$1,000 per employee
- Another troll sued cafés, hotels, and libraries for “providing Wi-Fi to customers”
- Chris Crawford (game developer) received a letter claiming a patent on “allowing users to move a character on screen using arrow keys”
- Nathan Myhrvold’s Intellectual Ventures collected 30,000+ patents and made billions by threatening lawsuits (called “the most hated company in tech” by many) → These are not inventors protecting their work – they are pure legal extortion using software and business-method patents.

Zoosk (Real-world patent example)

- Zoosk runs a dating website/app.
- Mobile devices connect to Zoosk’s central server.
- The server collects information from all phones (location, profile, etc.).
- Zoosk patented the idea that:
 - the server identifies users who are physically close to each other (same café, same event, etc.)
 - and actively invites/connects only those users who are “members of a same social network”
- Key claim: the server shares profile information of nearby users only with other nearby users who are already connected on the same social graph.

- They explicitly patented “they’re on Zoosk” + “they’re physically near each other” + “they’re already friends on another social network” → then automatically suggest or connect them.
- Result: A very specific but extremely common dating-app feature is now covered by a U.S. patent → any competitor who does almost the same thing risks being sued.

This is a perfect example of a “business-method + software” patent on a computer patent that many people consider obvious, yet it was granted.

Problem: Open Source is under attack, too!

- Chart shows number of patent lawsuits targeting open-source projects has been rising every year since 2005.
- Open-source projects and companies that use them (Red Hat, Google, Android manufacturers, etc.) are increasingly hit with patent demands.
- Right side news headlines:
 - “Patent troll attacks open source”
 - “New wave of patent attacks against Linux and open source”
- Even though open source is freely shared, patents can still block its use because patents cover ideas, not code.
- Famous cases: Microsoft vs TomTom (FAT filesystem patents), Apple vs HTC/Samsung (Android), countless trolls attacking Linux servers.

Open source is not safe from the patent problem — it is one of the biggest targets.

What next?

Possible solutions being discussed or already in use:

- Alternative licensing models
 - Defensive Patent License (DPL)
 - The Open Invention Network (OIN) – big cross-license pool
 - License on Transfer (LOT) – patent automatically becomes free if sold to a troll
- Bogus patent bounties (pay money to anyone who kills a bad patent)
- Unified Patents – companies pool money to kill bad patents together
- Commons reform (make it harder to get software patents)
- Abolish software patents completely (like New Zealand and India largely did)

“Project Jengo” by Cloudflare – they publicly offered cash rewards to anyone who could invalidate bad patents that trolls were using against them.

Summary:

	Public domain & equivalents	Permissive license	Copyleft (protective license)	Noncommercial license	Proprietary license	Trade secret
Description	Grants all rights	Grants use rights, forbids almost nothing (allows proprietization, license compatibility)	Grants use rights, forbids proprietization	Grants rights for noncommercial use only. May not be combined with copyleft.	Traditional use of copyright ; no rights need be granted	No information made public
Software	PD, CC0	BSD , MIT , Apache	GPL , AGPL	JRL , AFPL	proprietary software , no public license	private, internal software
Other creative works	PD, CC0	CC BY	CC BY-SA , Free Art License	CC BY-NC , CC BY-NC-SA	Copyright , no public license	unpublished

1. Public domain license

Public domain licenses have no restrictions on usage. The creator of the software can choose to donate the software to the public domain. Copyright doesn't apply and anyone can share, modify, commercialize, or relicense it with no repercussions.

Users can create personal and professional products using the software without financial compensation paid to the original designer.

Example: GIMP, an open-source editing software for graphic design that is completely free for users.

2. GNU lesser general public license

The Free Software Foundation released GNU Lesser General Public License (LGPL), a free, open-source license allowing developers to use a software component released under the LGPL into their own software without releasing the source code of their own components.

Example: 7-Zip, an LGPL software allowing users to compress files into one smaller file.

3. Permissive license

This is the most common type of open-source software license with a few restrictions on how the user can change or redistribute the software. While permissive licenses do involve copyright protection, they typically only require that the copyright information be filed once the software is distributed.

Example: The Apache License 2.0, a software allowing users to use and modify software products originally developed by Apache. There are specific requirements against redistribution and patent infringement.

4. Copyleft license

Copyleft licenses, also known as reciprocal licenses, are not as commercially friendly as other software license types. They have more restrictions than LGPL licenses. Copyleft licenses involve a reciprocal relationship. If a developer chooses to modify this software and then distribute their version, they must make it freely available to users. This allows users of their version to continue the dynamic, creating their own free software in the process.

Example: Audacity, a FOSS digital audio editing software, lets users copy, distribute, and edit the software's source code.

Copyleft is about the freedom of users. Copyright is about restricting what others can do with created works. Copyright may grant the right to copy, license, and perform an original work of art, but the publisher won't allow acts like copying a DVD and passing a copy along to a friend.

5. Proprietary license

This software license model is any software that has a copyright involving limits of use and distribution. Proprietary licenses are used by software developers who wish to claim ownership of their work and stay in control of who uses it. It is the most restrictive of the five types of software licenses.

Example: Microsoft Windows is an example of a proprietary license. Users cannot reverse engineer or distribute the software.

Copyleft [software licenses](#) are considered *protective* or *reciprocal* (in contrast with [permissive free software licenses](#)):^[2] they require that information necessary for reproducing and modifying the work be made available to recipients of the software program.^[citation needed] This information is most commonly in the form of [source code](#) files, which usually contain a copy of the license terms and acknowledge the authors of the code. Copyleft helps ensure everyone's rights to freely use the product but it prohibits owning, registering copyright and earning royalties from copyright.

